SE 2228: Analysis and Design of Algorithms

Review for Final

Prof.Dr. M. C .Okur

General Info

- Coverage: the whole semester
- About 15 questions
- Goal: doable in 90 minutes
- Cheat sheet: you are allowed one A4 hand written sheet

Final Exam: Study Tips

- Study tips:
  - Study each lecture
  - Study the homeworks and homework solutions
  - Study the midterm and quiz exams
- Re-make your previous cheat sheets
  - Think about what you should have had on it the first time…cheat sheets is about *identifying* important concepts , not including everything.

Analyzing Algorithms: Run Times,Complexity,Big-Oh..

- We are interested in *asymptotic analysis*:
  - Behavior of algorithms as problem size gets large
  - Constants, low-order terms don’t matter

Upper Bound Notation

- We say : InsertionSort's run time is $O(n^2)$
  - Properly we should say run time is *in* $O(n^2)$
  - Read O as "Big-O" (you'll also hear it as "order")
- In general a function
  - f(n) is O(g(n)) if there exist positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

Some Well-known Computational Problems

- Sorting
- Searching
- Finding shortest paths in a graph
- Finding minimum spanning trees
- Traveling salesman problem
- Topological Sorting
- Knapsack problem
- Game programming
- ……

Iterative / Recursive

- Nonrecursive(Iterative) Algorithm

```
ALGORITHM Factorial(n)
f ← 1
for i ← 1 to n do
    f ← f * i
return f
```

- Recursive Algorithm
  Need : Base condition
  Fn. calls itself by changing its Parameter until base cond.

```
ALGORITHM Factorial(n)
if n = 0
    return 1
else
    return Factorial(n − 1) * n
```

Algorithm Design Methods

- Brute Force
- Divide and Conquer
- Greedy
- Dynamic Programming
- ……

Brute Force

- A straightforward approach to solving a problem, usually based on problem statement and definitions of the concepts involved
- “Force” comes from using computer power not intellectual power.

  In short, “brute force” means “Just do it!”

Example : We want to compute a^n.

Brute force solution: multiply a by itself n times.

$a^n = a*a*\ldots*a$

Example simple search and sort algorithms : Sequential search,selection sort,buble sort,…

Also, matrix multiplication, brute force closest pairs…

Divide and Conquer

General idea: Breaking large problems into smaller subproblems

- Divide a problem into subprograms of the same kind; solve subprograms using the same approach, and combine partial solution (if necessary).

Three steps are involved:

- Divide the problem into several subproblems, perhaps of equal size
- Subproblems are solved, typically recursively
- The solutions to the subproblems are combined to get a solution to the original proble

Examples: Binary search,merge sort,quicksort…

Sorting Summary

- Insertion sort:

Essentially the same way you order anything in daily life.

Ex: Ordering playing cards on one hand.

Best Case: O(n),already sorted list.

Worst Case: $O(n^2)$, if given in reverse order.

Average Case: $O(n^2)$, still a quadratic running time.

→ A good option When the data sets are relatively small or nearly sorted

Sorting Summary

Quick sort:

- Divide-and-conquer:
  - Partition array into two subarrays around the pivot:
    - All of first subarray < all of second subarray
    - Sort each subarray recursively
  - No merge step is needed!
- O(n log n) best and average case
- Fast in practice
- $O(n^2)$ worst case
  - Naïve implementation: worst case on sorted input !
  - Best case when pivot is always in the exact middle.

Sorting Summary

Merge sort:

- Divide-and-conquer:
  - Split array in half
  - Recursively sort subarrays
  - merge step : Linear-time
  - O(n log n) worst best and average case
- Doesn't sort in place , we need to use extra storage

→The space complexity of Merge sort is O(n)

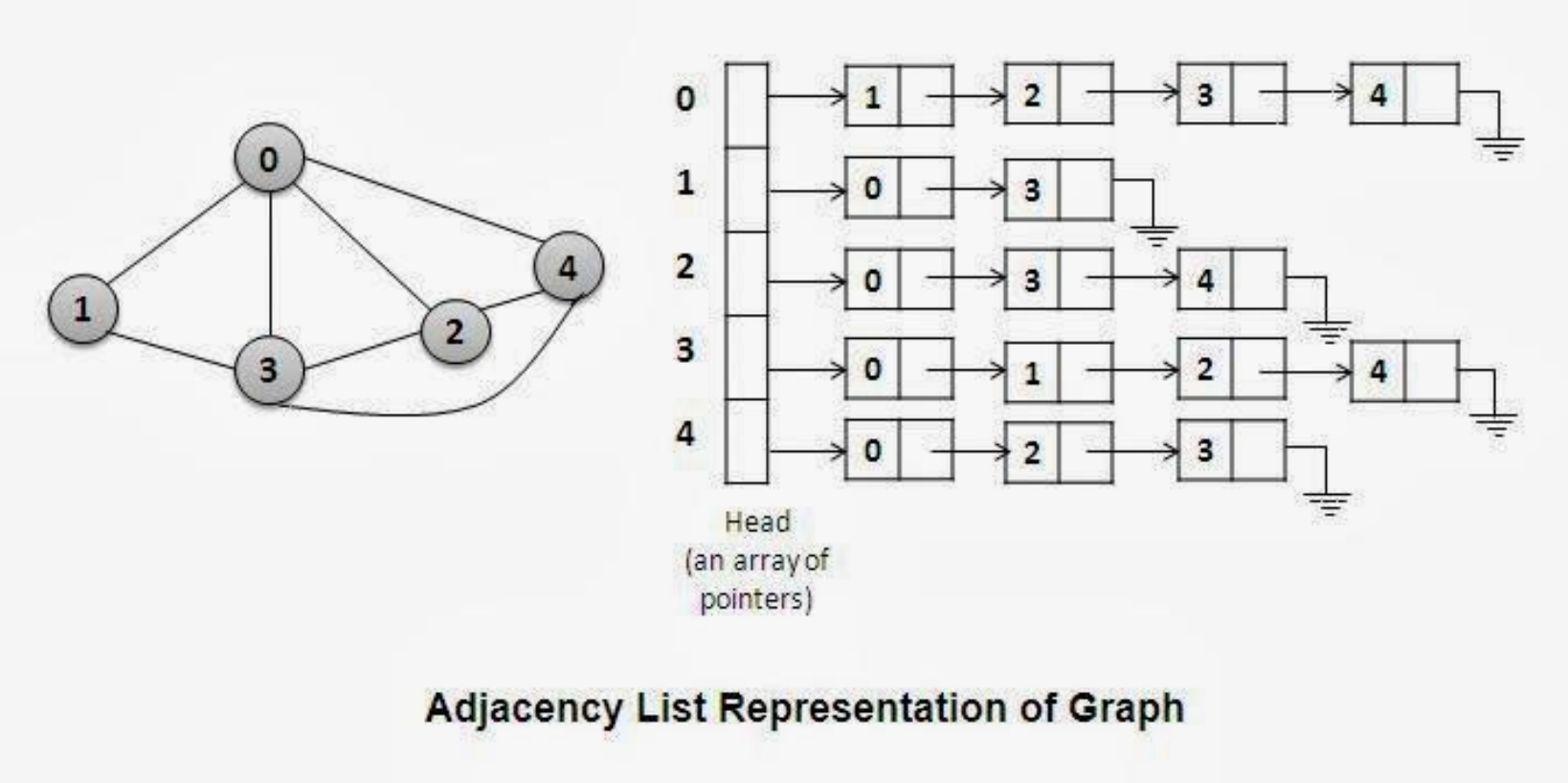
Sorting Summary

Heap sort:

- Uses the very useful heap data structure
  - Complete left justified binary tree
  - Heap property: parent key > children's keys(Maxheap)
- Sorts in place
- O(n) exchanges are needed.
- O(n log n) worst case

Graphs: Representations

- Graph Representation methods: adjacency matrix / adjacency linked list



Graphs : Search (Traversal)Methods

- Breadth-first search
- Depth-first search

Breadth First Search

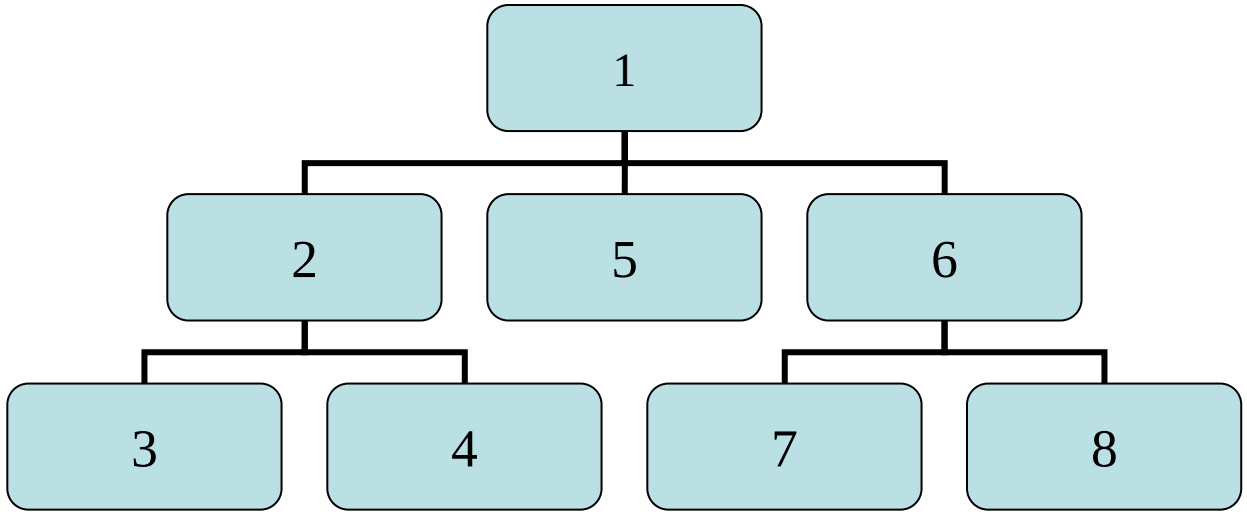
- Pick a source vertex S to start.
- Find and explore(visit) the vertices that are adjacent to S.
- Pick each child of S in turn and find the vertices adjacent to that child.
- Done when all children have been found and visited.
- This results in a tree that is rooted at the source vertex S.

Depth First Search

- Start at some source vertex S.
- Find (or explore) the first vertex that is adjacent to S.
- Repeat with this vertex down and explore the first vertex that is adjacent to it.
- When a vertex is found that has no unexplored vertices adjacent to it then backtrack up one level and continue
- Done when all children have been discovered and examined.
- Results in a forest of trees.

Depth First Search

- Example : DFS on binary tree. Specialized case of more general graph. The order of the search is down paths and from left to right.
  - The root is examined first; then the left child of the root; then the left child of this node, etc. until a leaf is found. At a leaf, backtrack to the lowest right child and repeat.



Greedy algorithms

- The greedy approach constructs a solution through a sequence of steps until a complete solution is reached, On each step, the choice made must be:
  - *Feasible:* Satisfy the problem's constraints
  - *locally optimal:* the best local choice
  - *Irrevocable:* Once made, it cannot be changed later
  - *Greedy choice property*: a locally optimal choice leads to a globally optimal solution

Greedy algorithms

Examples:

- change making
- Minimum Spanning Tree (MST)
- Single-source shortest paths
- simple scheduling problems
- Huffman codes

- Greedy approximations:
  - Traveling Salesman Problem (TSP)
  - Knapsack problem
  - Other combinatorial optimization problems

Single-Source Shortest Paths: Djkstra

- Uses optimal substructure property:

A problem is said to have optimal substructure if an optimal solution can be constructed efficiently from optimal solutions of its subproblems.

- What does Dijkstra's algorithm do?
  - Given a graph and a source vertex in graph, finds shortest paths from source to all vertices in the graph.
  - What is the running time?
    - Time Complexity is $O(|V|^2)$. If the input graph is represented using adjacency list, it can be reduced to

      O(E log V) with the help of binary heap
  - When does Dijkstra's algorithm not apply?
    - Dijkstra's algorithm doesn't work for graphs with negative weight edges

Spanning Tree and MST

Spanning Tree :

A spanning tree T of an undirected graph G is a subgraph that is a tree which includes all of the vertices of G with no cycles.

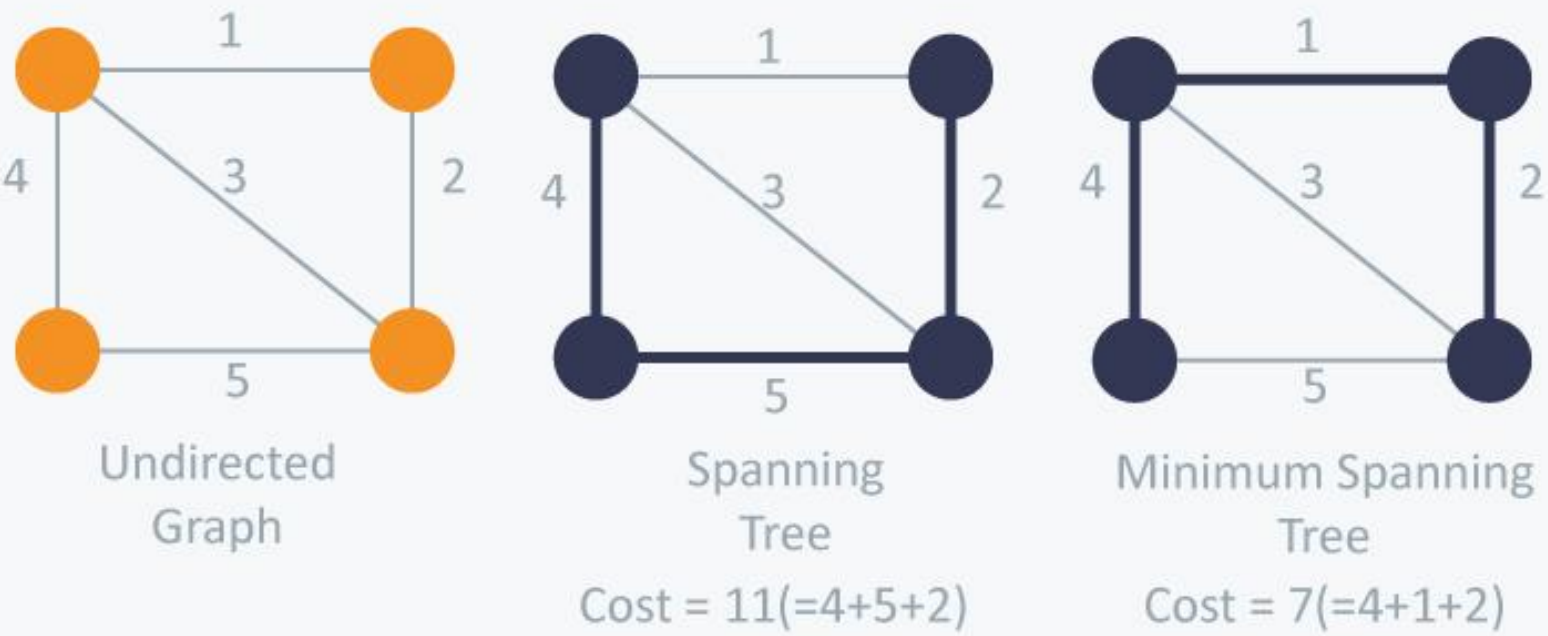
Minimum spanning tree:

A spanning tree whose total weight is minimum.

- What is a MST?
- Optimal substructure ?

Optimal substructure for MST: →At any stage, the subtree so far has minimum total weight.

Example


1
4
3
2
5
Undirected Graph
1
4
3
2
5
Spanning Tree
Cost = 11(=4+5+2)
1
4
3
2
5
Minimum Spanning Tree
Cost = 7(=4+1+2)

MST Algorithms

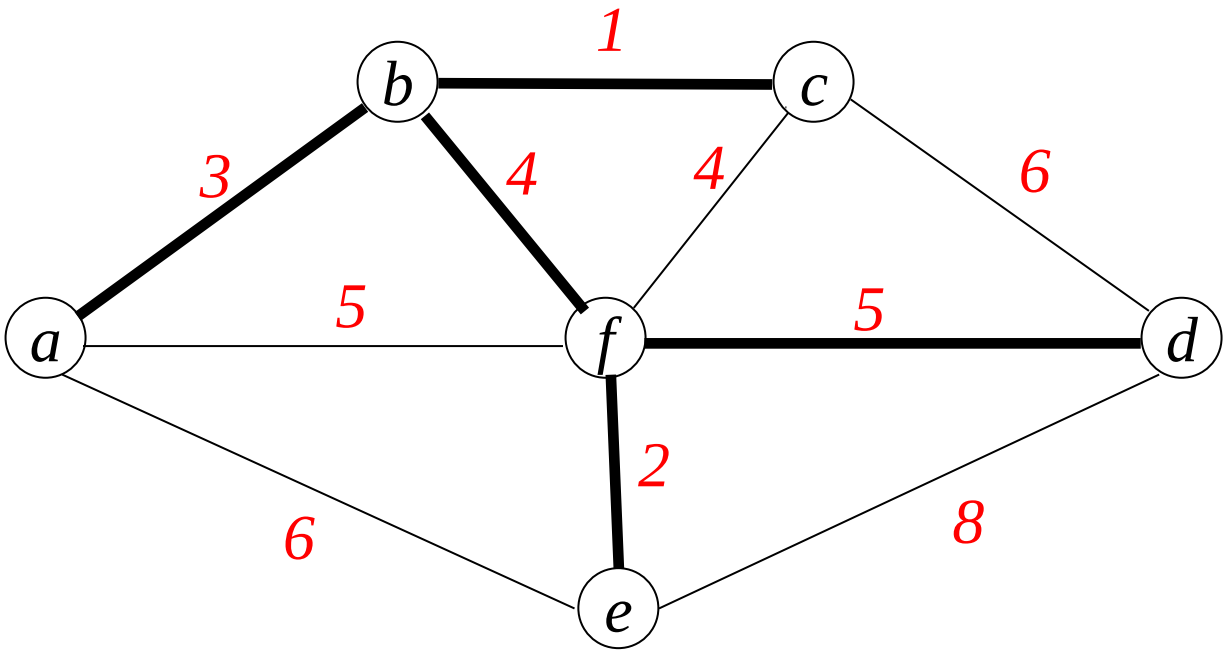
- Kruskal’s algorithm
- Prim’s algorithm
- Run times and complexities

Greedy algorithm for MST: Kruskal

- Start with empty forest of trees
- "Grow" MST one edge at a time
  - intermediate stages usually have forest of trees (not connected)
- At each stage add minimum weight edge among those not yet used that does not create a cycle
  - edges are initially sorted by increasing weight
  - at each stage the edge may:
    - expand an existing tree
    - combine two existing trees into a single tree
    - Avoid cycles
  - need efficient way of detecting/avoiding cycles
- Algorithm stops when all vertices are included

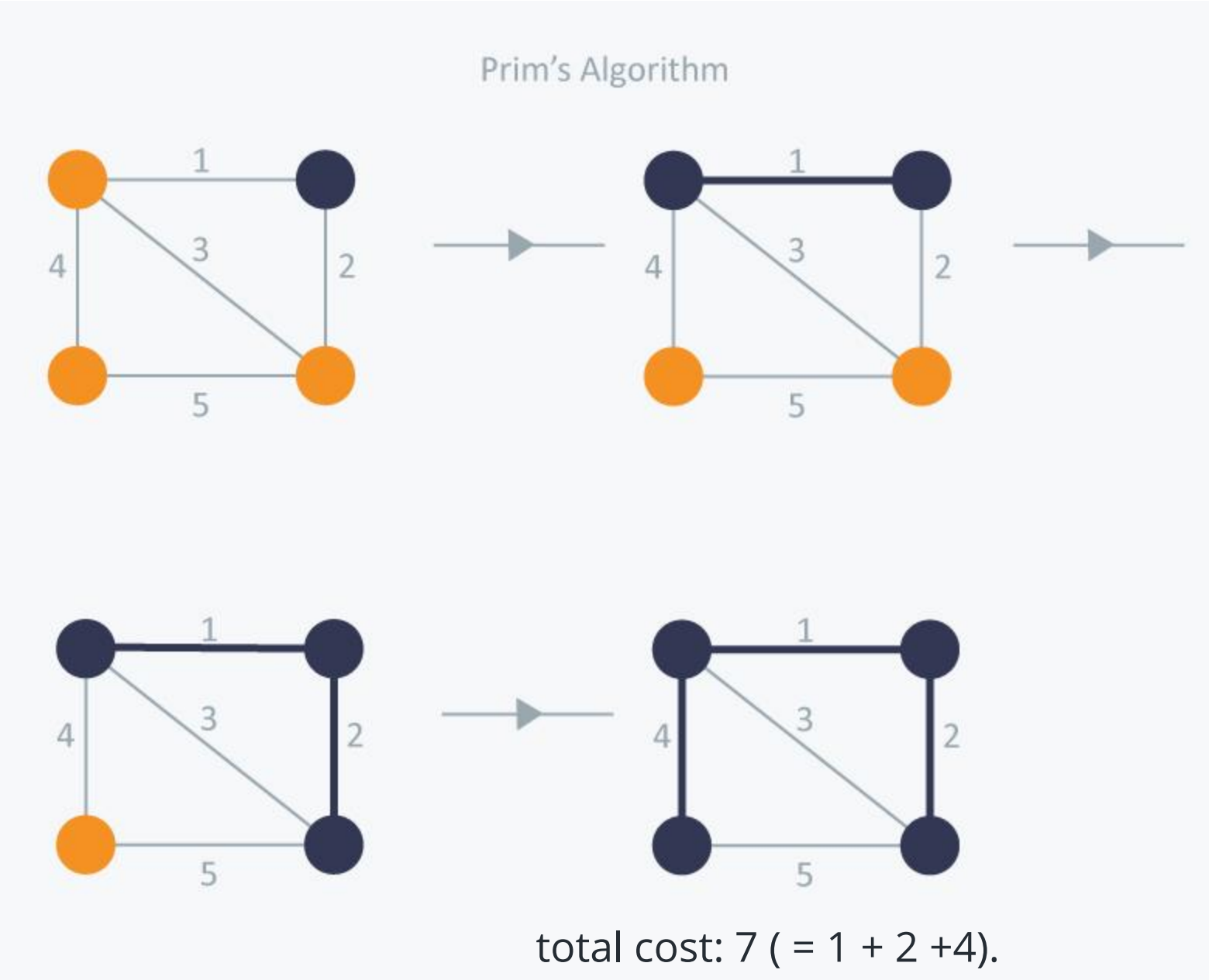
Kruskal : Example

bc	ef	ab	bf	~~cf~~	~~af~~	df	ae	cd	de
1	2	3	4	~~4~~	~~5~~	5	6	6	8



Greedy algorithm for MST: Prim

- Start with an arbitrary vertex and mark it. In each iteration mark a new vertex that is adjacent to the one that we have already marked.
- In each iteration,we have to choose a cheapest edge wihch does not create a cycle.



total cost: 7 (= 1 + 2 +4).

Topological Sort

- Topological sort
  - What is Topological Sort?
  - Algorithms
- Main use: Indicate order (schedule)of events, what should happen first?
- Algorithm for Topological-Sort:
  - Call DFS(G) to compute f(v), the finish time for each vertex.
  - As each vertex is finished insert it onto the front of the linked list.
  - Return the list.
- Time is Θ(V+E), the same complexity for DFS.
- The second algorithm : Source removal algorithm.

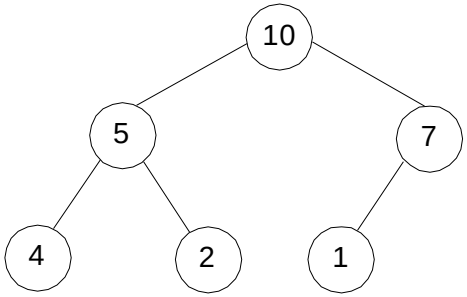
Heaps

- A *heap* is a “complete” binary tree, usually represented as an array.
- Two types are Maxheap and Minheap
- To represent a Maxheap as an array:

```
Parent(i) { return ⌊i/2⌋; }
Left(i) { return 2*i; }
right(i) { return 2*i + 1; }
```

Heap Implementation

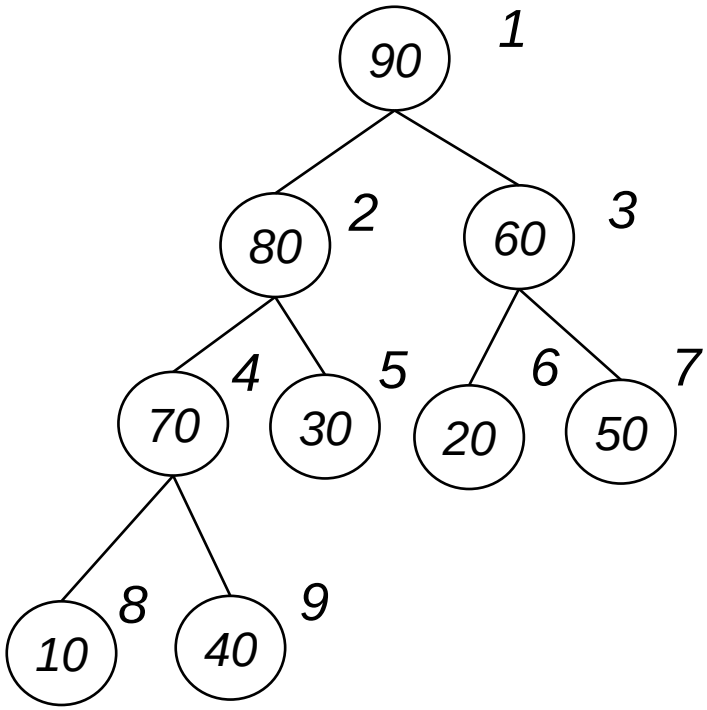
A maxheap and its array representation :



Index	1	2	3	4	5	6
value	10	5	7	4	2	1

Example :Representation of a Maxheap

- Maxheap as a binary tree:



- *Array representation :*

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]
	90	80	60	70	30	20	50	10	40	-	-	-

Heap Operations

- **`Heapify()`**: maintain the heap property.
- **`BuildHeap()`**: build heap bottom-up by running **`Heapify()`** on successive subarrays.
- **`Heapsort()`**: An in-place sorting algorithm.

Heapsort

- Two Stage algorithm to sort a list of *n* keys
- First, heap construction : $O(n)$
- Second, sequential root deletion (the largest is deleted first, and the second largest one is deleted second, etc …)
- Therefore, the time efficiency of heapsort is O(nlogn) in the worst case, which is the same as quicksort and mergesort .
- Note: Average case efficiency is also the same.

Priority Queues

- The heap data structure is often used for implementing *priority queues*
  - A data structure for maintaining a set *S* of elements, each with an associated value or *key*
  - Supports the operations **`Insert()`**, **`Maximum()`**, and **`ExtractMax()`**
  - Commonly used for scheduling and event simulation

Design Strategy : Dynamic Programming

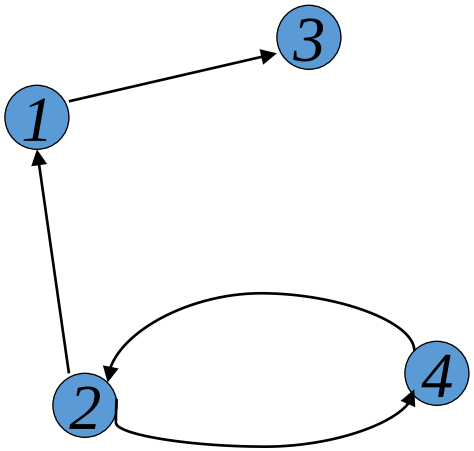
- Dynamic Programming is a general algorithm design technique
- "Programming" here means "planning"
- Main idea:
  - solve several smaller (overlapping) subproblems
  - record (memoize)solutions in a table, so that each subproblem is only solved once
  - final state of the table will be (or contain) the solution

Dynamic Programming

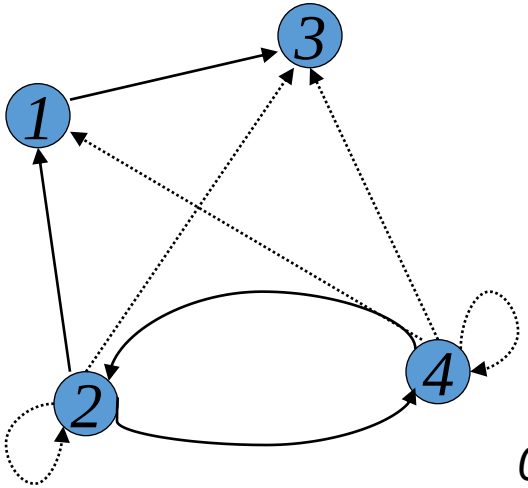
- Indications: optimal substructure, repeated subproblems
  - *Dynamic programming*: build table of subproblem solutions bottom-up
- Example problems : All pairs shortest paths:Floyd-Warshall Algorithm
- 0-1 Knapsack problem

Warshall's Algorithm :Transitive Closure

- Computes the transitive closure of a relation
- (Alternatively: all paths in a directed graph)
- Example of transitive closure:



$$\begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{matrix}$$



$$\begin{matrix} 0 & 0 & 1 & 0 \\ 1 & \mathbf{1} & \mathbf{1} & 1 \\ 0 & 0 & 0 & 0 \\ \mathbf{1} & 1 & \mathbf{1} & \mathbf{1} \end{matrix}$$

Hashing

- Motivation: symbol tables
  - A compiler uses a *symbol table* to relate symbols to associated data
    - Symbols: variable names, procedure names, etc.
    - Associated data: memory location, call graph, etc.
  - For a symbol table (also called a *dictionary*), we care about search(axess), insertion, and deletion.
  - We typically don’t care about sorted order

Choosing A Hash Function

- Choosing a good hash function is crucial
  - Bad hash function puts all elements in same slot
  - A good hash function:
    - Should distribute keys ~ uniformly into slots
    - Should not depend on patterns in the data
- We discussed some methods:
  - Division method
  - Multiplication method
  - Mid square method
  - Hash fn for non-numeric keys

Hash Tables

- More formally:
  - Given a table *T* and a record *x*, with key (= symbol) and other data, we need to support:
    - Insert (*T*, *x*)
    - Delete (*T*, *x*)
    - Search(*T*, *x*)
  - Don't care about sorting the records
- *Hash tables* support all the above in O(1) expected time
- An important problem of hashing is collision.

Resolving Collisions

- *How can we solve the problem of collisions?*
- *Linear Probing (Open addressing)*
  - To insert: if slot is full, try another slot, and another, until an open slot is found (*probing*)
  - To search, follow the same sequence of probes as would be used when inserting the element
- Quadratic probing :In case of collisions try next slots i^2; i=1,2,3,…
- *Chaining*
  - Keep linked list of elements in slots
  - Upon collision, just add new element to list
- Rehashing : In case of a collision use a second (or more) hash function

P, *NP*, and *NP*-Complete Problems

- As we discussed, problems that can be solved in polynomial time are usually called tractable and the problems that cannot be solved in polynomial time are called intractable, now
- *Is there a polynomial-time algorithm that solves the problem?*
- *P*: the class of decision problems that are solvable in $O(p(n))$, where $p(n)$ is a polynomial on n
- *NP*: the class of decision problems that are solvable in polynomial time on a *nondeterministic* machine
- A decision problem D is *NP*-complete iff
  1. $D \in NP$
  2. Every problem in *NP* is polynomial-time reducible to D

P,NP,NP-Completeness

- What do we mean when we say a problem is in ***P***?
  - Ans: A solution can be found in polynomial time
- *What do we mean when we say a problem is in **NP**?*
  - Ans: A solution can be verified in polynomial time
- What is NP-Complete ?
  - Ans : All problems are NP and can be reduced(Converted) to each other.
- *What is the relation between **P** and **NP**?*
  - Ans: **P** $\subseteq$ **NP**, but no one knows whether **P = NP**

Learn your subjects well and do good in the exams.

Good Luck !